

Python Weekly Assignment

Topics: Strings · Data Structures · Functions and Modules

Section A — Strings

1. Write a function `analyze_sentence(sentence)` that takes a sentence and returns a dictionary with `word_count`, `char_count` (excluding spaces), `unique_words` (case-insensitive), and `most_frequent_word`. Test it with:

```
"The quick brown fox jumps over the lazy dog the fox"
```

2. Given the string below, do the following and print each result:

```
text = "A man a plan a canal Panama"
```

- a) Reverse the string without using slicing.
- b) Check if it is a palindrome, ignoring case and spaces.
- c) Replace every vowel with *.
- d) Count how many times each vowel appears.

3. Write a function `format_report(data)` that takes a list of tuples (name, score, grade) and prints a formatted table where name is left-aligned (width 15), score is right-aligned (width 6), and grade is centered (width 8). Use this data:

```
data = [("Alice", 92, "A"), ("Bob", 78, "B+"), ("Charlie", 55, "C"), ("Diana", 100, "A+")]
```

Section B — Data Structures

4. Write a function `list_ops(nums)` that:

- a) Removes duplicates while preserving the original order.
- b) Returns a new list with only elements at even indices, squared.
- c) Flattens the nested list into a single list using list comprehension.

```
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5] nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
```

5. Given the tuple below:

```
employee = ("E042", "Riya Sharma", "Data Science", 72000, "Pune")
```

- a) Unpack it into meaningful variable names.
- b) Use * unpacking to separate the first element, the last, and everything in between.
- c) Write a function `update_salary(emp_tuple, new_salary)` that returns a new tuple with the salary updated.

6. Using the sets below, find:

```
batch_a = {"Aarav", "Priya", "Karan", "Meera", "Sahil"} batch_b = {"Karan", "Meera", "Tanvi", "Rohan"} batch_c = {"Priya", "Aarav"}
```

- a) Students present in both batches.
- b) Students in `batch_a` but not in `batch_b`.

c) All unique students across both batches.

d) Whether batch_c is a subset of batch_a.

7. Write a function `inventory_manager(records)` that takes a list of tuples (product, category, quantity, price) and returns a dictionary grouped by category. Each category should have `total_items`, `total_value`, and a list of product names. Use:

```
records = [ ("Laptop", "Electronics", 10, 55000), ("Mouse", "Electronics", 50, 499), ("Desk", "Furniture", 8, 12000), ("Chair", "Furniture", 15, 4500), ("Pen", "Stationery", 200, 20), ]
```

Section C — Functions and Modules

8. Implement the following:

a) A function `power_all(*args, exp=2)` that returns a list of each number raised to `exp`.

b) A function `profile(**kwargs)` that prints each key-value pair as `Key : Value`.

c) Use a lambda inside `sorted()` to sort the list of dicts below by `cgpa` in descending order.

```
students = [{"name": "Arjun", "cgpa": 8.4}, {"name": "Sanya", "cgpa": 9.1}, {"name": "Dev", "cgpa": 7.8}]
```

9. Create a module `stats_utils.py` with three functions:

a) `mean(data)` — arithmetic mean.

b) `variance(data)` — population variance without using the statistics module.

c) `normalize(data)` — min-max normalization returning values between 0 and 1.

In a separate `main.py`, import and use these functions on the dataset below. Also compute the standard deviation using the `math` module.

```
dataset = [12, 45, 7, 23, 56, 89, 34, 67, 11, 90]
```

10. Complete the following:

a) Write a closure `make_multiplier(n)` that returns a function multiplying its input by `n`. Create triple and quintuple using it.

b) Write a decorator `@timer` that prints the execution time of any function it wraps. Apply it to `slow_sum(n)` which computes the sum of squares from 1 to `n`.

c) Write a decorator `@validate_positive` that raises a `ValueError` if any argument is zero or negative. Apply it to `safe_sqrt(x)`.